

Mediapipe Palm Follow (PTZ)

1. Experiment Objective

PTZ pan-tilt + car body collaborative palm following — MediaPipe Hands detects the palm, locks onto the index finger MCP (landmark 5), and the body turns to home based on s1 + follows distance based on the palm bbox size.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Following	Pan-tilt tracking + coordinated body movement; the car actively turns and moves forward/back to follow
MediaPipe Hands	Google's open-source hand detection framework; 21 landmarks, with the index finger MCP (landmark 5) as the tracking anchor
PD Control	Proportional-derivative control; a dual loop of PTZ 20 Hz + body 10 Hz
Error sharing (body_share)	The body takes on the pan-tilt angle $\times$ body_share homing task
Angular-rate feedforward (body_ff_gain)	When the body rotates, the pan-tilt compensates in the opposite direction
Distance Follow	Deviation between the palm bbox size and <code>target_hand_size_px</code> (150 px) $\rightarrow$ PD controls the linear velocity
Lost-frame tolerance	The body is frozen only after N consecutive lost frames (lost_frame_threshold=5)

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
└─ hand_follow_ptz_node
    └─ MediaPipe Hands → 21 landmarks → index finger MCP (landmark 5) + bbox size
        └─ PTZ PD 20Hz → normalized error → PD → step limiting → /cmd_ptz_angle
            └─ Body PD 10Hz
                └─ s1  $\times$  body_share → PD → /cmd_vel.angular.z (turning)
                    └─ palm bbox size → distance PD → /cmd_vel.linear.x (forward/back)
```

A MediaPipe Python environment is required (ROS2 and the workspace environment).

3. Core Topics Quick Reference

Firmware Uplink (data upstream):

Topic Name	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	BEST_EFFORT, KEEP_LAST depth=1, VOLATILE	Raw image stream from the ESP32 camera
/odom	nav_msgs/Odometry	BEST_EFFORT	Odometry data (published by the state estimation layer)

Firmware Downlink (command downstream):

Topic Name	Message Type	QoS	Field	Range	Description
/cmd_ptz_angle	geometry_msgs/Vector3	BEST_EFFORT, KEEP_LAST depth=1	x (horizontal)	-90°~90°	s1 horizontal angle
			y (tilt)	-90°~20°	s2 tilt angle
/cmd_vel	geometry_msgs/Twist	BEST_EFFORT, KEEP_LAST depth=1	angular.z	±0.12 rad/s	Turning angular velocity
			linear.x	±0.12 m/s	Forward/back linear velocity

ROS2 Internal (internal communication):

Topic Name	Message Type	QoS	Description
/joint_states	sensor_msgs/JointState	RELIABLE	Joint states published by the state estimation layer

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑
No-PTZ version	✗
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

2. Start Agent + Camera + Joint Model (Terminal 1)

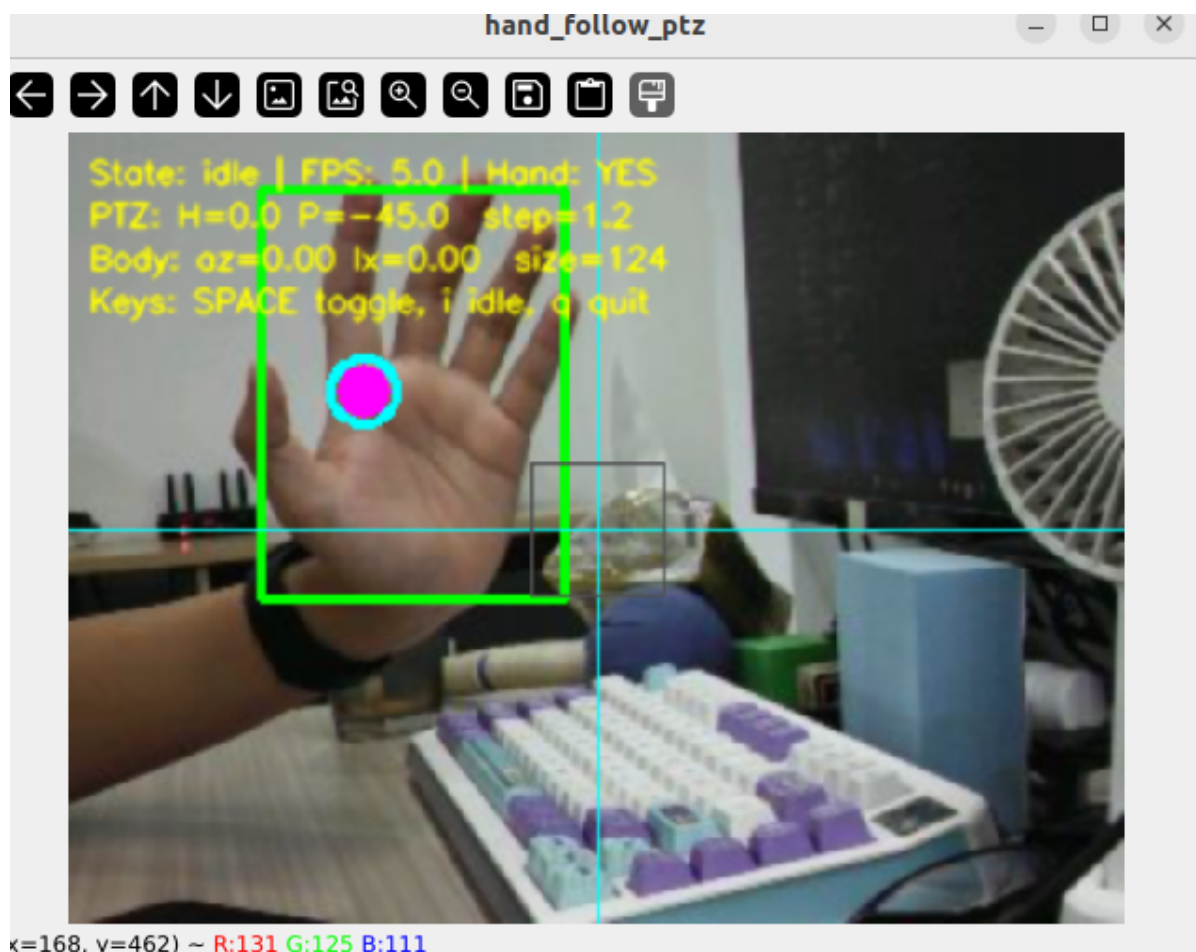
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Palm Follow (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg hand_follow_ptz.launch.py
```

After startup, the frame shows the `idle` state; palm detection works normally but the pan-tilt and body do not move.



4. Operation Instructions

(1) Workflow

Uses the same three-layer coordinated control framework as Color Follow, with MediaPipe Hands detection replacing HSV detection.

MediaPipe palm detection: image conversion → scaled to 320×240 →

`HandDetector.findHands()` detects 21 landmarks. The index finger MCP (landmark 5) is taken as the tracking anchor, and the min/max of all landmarks compute the bbox size.

Three-layer coordinated control:

- **PTZ PD (20 Hz):** index finger MCP pixel error → PD (`pd_kp`=0.7, `pd_kd`=0.25) → feedforward compensation → `/cmd_ptz_angle`
- **Body angular velocity (10 Hz):** $s1 \times \text{body_share}$ (0.5) → PD (`angular_kp`=0.05) → `/cmd_vel.angular.z`
- **Distance follow (10 Hz):** deviation between the palm bbox and `target_hand_size_px` (150 px) → PD → `/cmd_vel.linear.x`

Lost-frame tolerance and fist handling: no hand for ≥ 5 consecutive frames → freeze the body. A fist in TRACKING outputs nothing but keeps the PD state.

(2) Interaction

Key	Function
Space	idle ↔ tracking
i / I	idle, stop
q / Q / ESC	Exit

(3) Safety and Edge Cases

- No hand ≥ 5 frames → freeze the body
- Pan-tilt limiting: horizontal $[-90^\circ, 90^\circ]$, tilt $[-90^\circ, 20^\circ]$
- Velocity limiting: angular velocity ± 0.12 rad/s, linear velocity ± 0.12 m/s
- Stop first on exit + `os._exit(0)`

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **idle state:** The frame streams in real time; palm detection works normally but the pan-tilt and body are stationary, and the state shows `idle`
- **tracking + palm present:** The pan-tilt locks onto the index finger MCP, the body turns with `s1`, and distance following maintains a palm size of about 150 px
- **tracking + fist:** Tracking pauses but the PD is kept; it resumes seamlessly when the palm opens
- **Palm lost ≥ 5 frames:** the body freezes and stops moving
- **Palm too close:** the car moves back; **palm too far:** the car moves forward
- **Palm within the deadband:** the pan-tilt step is zeroed, completely still without jitter

5. Program Analysis

Source code paths:

- Launch file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/hand_follow_ptz.launch.py`
- Node file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/hand_follow_ptz_node.py`
- Hand detection module:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/hand_detector.py`

1. Core Node Analysis

The node `HandFollowPtzNode` inherits from `rc1py.node.Node`, detects palm landmarks with MediaPipe Hands, uses the index finger base (landmark 5) as the anchor, and implements complete follow control through the PTZ PD + body dual-loop PD.

Constructor `__init__` core logic:

```
def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._image_lock = threading.Lock()
    self._track_lock = threading.Lock()

    self._state = STATE_IDLE          # minimal state machine: idle + tracking
    self._shutdown = False

    self._has_hand = False
    self._has_target = False
    self._lost_frames = 0
    self._finger_count = 0
    self._target_x: Optional[float] = None
    self._target_y: Optional[float] = None
    self._target_size_px: Optional[float] = None

    self._h_deg = 0.0                 # initial s1 horizontal angle
    self._p_deg = -45.0               # initial s2 tilt angle (45° downward)

    # PD control history state
    self._prev_err_x = 0.0
    self._prev_err_y = 0.0
    self._prev_ctrl_t = 0.0
    self._last_sx = 0.0
    self._last_sy = 0.0

    self._prev_s1 = 0.0
    self._prev_s1_eff = 0.0
    self._prev_body_t = 0.0
    self._last_body_az = 0.0

    self._prev_size_err = 0.0
    self._prev_linear_t = 0.0
```

```

self._declare_params()
self._load_params()

self._h_deg = 0.0
self._p_deg = -45.0

# Initialize the MediaPipe Hands detector
self._hand_det = HandDetector()

# Image subscription (QoS: BEST_EFFORT, KEEP_LAST depth=1, VOLATILE)
video_qos = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST, depth=1,
    reliability=QoSReliabilityPolicy.BEST_EFFORT,
    durability=QoSDurabilityPolicy.VOLATILE,
)
self._img_sub = self.create_subscription(
    Image, self._img_topic, self._on_image, video_qos
)
self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)
self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, 10)

# Three independent timers: image processing, PTZ control, body control
self._img_timer = self.create_timer(
    1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image)
self._ptz_timer = self.create_timer(
    1.0 / max(float(self._ptz_rate), MINIMUM_TIMER_RATE_HZ), self._ptz_tick)
self._body_timer = None
self._rebuild_body_timer(False)

# Publish the initial PTZ angle
self._pub_ptz()

```

Body control callback `_body_tick` (including turning + distance following):

```

def _body_tick(self) -> None:
    if self._state != STATE_TRACKING:
        self._reset_body_pd()
        self._pub_zero_cmd()
        return

    with self._track_lock:
        has_target = self._has_target
        size_px = self._target_size_px

    if not has_target:
        self._reset_body_pd()
        self._pub_zero_cmd()
        return

    s1 = float(self._h_deg)
    if abs(s1) < self._ang_tol_deg:
        s1 = 0.0
    s1_eff = s1 * self._body_share

    n = time.perf_counter()
    dt_body = n - self._prev_body_t if self._prev_body_t > 0.0 \

```

```

        else 1.0 / max(float(self._body_rate), MINIMUM_TIMER_RATE_HZ)
dt_body = max(dt_body, 1e-3)
ds1_eff = (s1_eff - self._prev_s1_eff) / dt_body

# Angular-velocity PD: s1_eff × kp + derivative × kd
az = self._body_kp * s1_eff + self._body_kd * ds1_eff
az = clamp(az, -self._max_angular_z, self._max_angular_z)
if self._rev_body:
    az = -az

cmd = Twist()
cmd.angular.z = float(az)

linear_x = 0.0
if self._enable_distance and size_px is not None:
    size_err = float(self._target_hand_size_px - size_px)
    if abs(size_err) < self._distance_tol_px:
        size_err = 0.0
    dt_linear = n - self._prev_linear_t if self._prev_linear_t > 0.0 \
        else dt_body
    dt_linear = max(dt_linear, 1e-3)
    d_err = (size_err - self._prev_size_err) / dt_linear
    linear_x = self._distance_kp * size_err + self._distance_kd * d_err
    linear_x = clamp(linear_x, -self._max_linear_x, self._max_linear_x)
    if self._rev_linear:
        linear_x = -linear_x
    self._prev_size_err = size_err
    self._prev_linear_t = n
else:
    self._prev_size_err = 0.0
    self._prev_linear_t = 0.0

cmd.linear.x = float(linear_x)
self._cmd_pub.publish(cmd)

self._prev_s1 = s1
self._prev_s1_eff = s1_eff
self._prev_body_t = n
self._last_body_az = az

```

`_body_tick` control chain:

```

TRACKING + has_target
→ s1 = h_deg (current PTZ horizontal angle)
→ s1 × body_share (50%) = s1_eff
→ angular-velocity PD (kp=0.05, kd=0.05) → /cmd_vel.angular.z (limited ±0.12 rad/s)
→ palm bbox size size_px
→ target_hand_size_px (150 px) - size_px = size_err
→ deadband distance_tolerance_hand_size_px (12 px) check
→ distance PD (kp=0.008, kd=0.003) → /cmd_vel.linear.x (limited ±0.12 m/s)

```

2. Key Parameters

Parameter definition file:

```
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/hand_follow_ptz.launch.py
```

Parameter	Type	Default Value	Description
<code>pd_kp</code> / <code>pd_kd</code>	float	<code>0.7</code> / <code>0.25</code>	Pan-tilt PD gains
<code>ptz_control_rate</code>	float	<code>20.0</code> Hz	Pan-tilt control rate
<code>body_control_rate</code>	float	<code>10.0</code> Hz	Body control rate
<code>body_share</code>	float	<code>0.5</code>	Ratio of the error taken on by the body
<code>body_ff_gain</code>	float	<code>1.3</code>	Angular-rate feedforward gain
<code>enable_distance_follow</code>	bool	<code>true</code>	Enable distance following
<code>target_hand_size_px</code>	float	<code>150.0</code> px	Distance-follow target palm size
<code>distance_tolerance_hand_size_px</code>	float	<code>12.0</code> px	Distance tolerance deadband
<code>distance_kp</code> / <code>distance_kd</code>	float	<code>0.008</code> / <code>0.003</code>	Distance PD gains
<code>max_angular_z</code>	float	<code>0.12</code> rad/s	Maximum turning angular velocity
<code>max_linear_x</code>	float	<code>0.12</code> m/s	Maximum linear velocity
<code>lost_frame_threshold</code>	int	<code>5</code>	Lost-frame tolerance threshold

6. Frequently Asked Questions

Problem	Cause	Solution
Palm loss triggers freezing	Quickly flipping the palm	Keep the palm plane facing the camera
Palm not recognized	MediaPipe environment not activated	First run <code>source /opt/ros/humble/setup.bash</code> <code>source ~/microros_ws/install/setup.bash</code>
Distance-follow overshoot	PD gain too large	Reduce <code>distance_kp</code>

